

Go Fundamentals for Distributed Algorithms II

Danilo Kaćanski

Faculty of Technical Sciences, Novi Sad
Applied Algorithms in Control Systems

March 2026.

Control Flow

Age classification

```
age := 17
```

```
if age < 13 {  
    fmt.Println("child")  
} else if age < 20 {  
    fmt.Println("teenager")  
} else {  
    fmt.Println("adult")  
}
```

- Conditions evaluate to **boolean values**
- Only one branch is executed

Key Idea

Go uses standard conditional branching, but conditions do not need parentheses.

if with_INITIALIZER

Initializer inside if

```
func isEven(x int) bool {  
    return x%2 == 0  
}  
  
if ok := isEven(8); ok {  
    fmt.Println("even")  
} else {  
    fmt.Println("odd")  
}
```

- The variable `ok` exists only inside the if/else chain
- Useful for short-lived helper values

Key Idea

The initializer is scoped to the whole if/else statement, not outside it.

Scope and Code Blocks

Outer and inner scope

```
var a = 10 // package-level variable
func main() {
    fmt.Println(a) // prints 10 (outer a)
    {
        a := 15 // NEW variable (shadows outer a)
        fmt.Println(a) // prints 15 (inner a)
    }
    fmt.Println(a) // prints 10 (outer a again)
}
```

- Inner blocks create a new scope
- Inner `a` hides the outer one

Key Idea

Variables with the same name can refer to different values in different scopes.

Package-Level Scope

- Variables declared outside functions are **package-level**
- All files in the same package can access package-level symbols
- This is useful for shared constants, helpers, and global configuration

Important

Package-level scope is broader than block scope, but still limited to the package.

Key Idea

Go organizes visibility through scopes: block, function, package, and exported package members.

for Loop Forms

Common forms

```
for i := 0; i < 3; i++ {  
    fmt.Println(i)  
}  
  
x := 0  
for x < 3 {  
    x++  
}  
  
for {  
    break  
}
```

- Go has only one loop keyword: `for` which covers classic, condition-only, and infinite loops

Key Idea

Go replaces while-style loops with different forms of `for`.

break and continue

Common forms

```
for i := 0; i < 5; i++ { // loop over values: 0,1,2,3,4
    if i == 2 {
        continue // skip this iteration 2 will NOT be printed
    }
    if i == 4 {
        break // stop the loop entirely exit before printing 4
    }
    fmt.Println(i) // prints current value of i
}
```

- continue skips the current iteration
- break exits the nearest loop

Key Idea

Use continue to skip work, and break to stop the loop.

Labeled break

Exit both loops

```
outerLoop:
for i := 0; i < 3; i++ {
    for j := 0; j < 3; j++ {
        if i == 1 && j == 1 {
            break outerLoop
        }
        fmt.Println(i, j)
    }
}
```

- Plain break exits only the inner loop
- Labeled break exits the chosen outer loop directly

Key Idea

Labeled break is cleaner than using flags when escaping nested loops.

goto (Controlled Use)

Local jump

```
func main() {  
    x := 1  
    if x == 1 {  
        goto done  
    }  
    fmt.Println("unreachable")  
done:  
    fmt.Println("finished")  
}
```

- goto jumps to a label in the same function
- Used rarely, structured control flow is preferred

Key Idea

goto exists in Go, but should be limited to simple, local control flow.

Exercise 3

Task 1: If Initializer

Task

Write a function that checks if a number is even.
Use it inside an `if` initializer:

- Print "even" or "odd"
- Try to use the variable outside the `if` block
- Why it fails?

Task 2: Shadowing vs Reassignment

Task

- Declare variable `a := 10`
- Inside a block:
 - First use `a := 20`
 - Then try `a = 30`
- Print values before, inside, and after the block

Task 3: Loop Control

Task

- Loop from 0 to 10
- Skip all even numbers using `continue`
- Stop the loop when number is greater than 7 using `break`
- Print remaining numbers

Task 4: Labeled Break

Task

- Create nested loops (i and j from 0 to 3)
- Print all pairs (i, j)
- Stop all loops when:

$$i == 2 \text{ and } j == 1$$

- First try using normal break
- Then fix using **labeled break**

Task 5: goto

Task

- Create a loop from 0 to 5
- When value is 3:
 - Jump to a label using goto
- Print "Skipped to end"